

Raindrop: 面向硬件设计的分组密码算法*

李永清¹, 李木舟^{1,2}, 付勇^{1,2}, 樊燕红^{1,2}, 黄鲁宁^{1,2}, 王美琴^{1,2}

1. 密码技术和信息安全教育部重点实验室, 青岛 266237

2. 山东大学 网络空间安全学院, 青岛 266237

通信作者: 王美琴, E-mail: mqwang@sdu.edu.cn

摘要: 我们设计了一个新的分组密码算法—Raindrop 分组密码算法. Raindrop 算法共有 Raindrop-128/128/60、Raindrop-128/256/80 和 Raindrop-256/256/100 三个版本. Raindrop 算法整体采用 Feistel 结构, 保证加解密的一致性. 轮函数采用 S 盒代换、行混合、列循环移位操作. S 盒由 Keccak 函数非线性运算的布尔表达式生成; 行混合操作中使用了深度为 1 的 4×4 二元域矩阵; 密钥生成算法采用异或数较少的线性操作; 通过采用简单的非线性层、线性层及密钥生成算法, 使得 Raindrop 分组算法在硬件实现上具有较好的优势. 在安全方面, 我们利用自动化工具 STP 对 Raindrop 分组算法进行评估, 评估结果显示, Raindrop 算法可以有效地抵抗差分攻击和线性攻击等已知攻击方法.

关键词: 分组密码算法; Feistel 结构; 面向硬件设计; 安全分析

中图分类号: TP309.7 **文献标识码:** A **DOI:** 10.13868/j.cnki.jcr.000342

中文引用格式: 李永清, 李木舟, 付勇, 樊燕红, 黄鲁宁, 王美琴. Raindrop: 面向硬件设计的分组密码算法[J]. 密码学报, 2019, 6(6): 803-814.

英文引用格式: LI Y Q, LI M Z, FU Y, FAN Y H, HUANG L N, WANG M Q. Raindrop: A block cipher designed for hardware[J]. Journal of Cryptologic Research, 2019, 6(6): 803-814.

Raindrop: A Block Cipher Designed for Hardware

LI Yong-Qing¹, LI Mu-Zhou^{1,2}, FU Yong^{1,2}, FAN Yan-Hong^{1,2}, HUANG Lu-Ning^{1,2}, WANG Mei-Qin^{1,2}

1. Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Qingdao 266237, China

2. School of Cyber Science and Technology, Shandong University, Qingdao 266237, China

Corresponding author: WANG Mei-Qin, E-mail: mqwang@sdu.edu.cn

Abstract: A new block cipher family named Raindrop was designed, which has three versions: Raindrop-128/128/60, Raindrop-128/256/80, and Raindrop-256/256/100. Raindrop cipher uses Feistel

* 基金项目: 国家自然科学基金 (61572293, 61502276, 61692276); 国家密码发展基金 (MMJJ20170102); 山东省重大科技创新工程 (2017CXGC0704); 山东省自然科学基金 (ZR2016FM22)

Foundation: National Natural Science Foundation of China (61572293, 61502276, 61692276); National Cryptography Development Fund (MMJJ20170102); Major Scientific and Technological Innovation Projects of Shandong Province, China (2017CXGC0704); Natural Science Foundation of Shandong Province, China (ZR2016FM22)

收稿日期: 2019-11-17 定稿日期: 2019-12-10

structure to make its encryption and decryption use the same process. The round function of Raindrop includes Substitution, Mix Row, and Cyclic Shift in Column. More precisely, the S-boxes are generated by the Boolean expression which is used in the nonlinear part of Keccak hash function. The Mix Row operation uses a 4×4 binary matrix whose depth is one. Linear operations and several XOR operations are used in the key schedule. By using a simple nonlinear layer, a simple linear layer and a simple key schedule, Raindrop has an advantage on hardware implementation. The automatic searching tool STP was used to evaluate the security of Raindrop cipher, and the results show that Raindrop is secure against all known attacks, such as differential attack and linear attack.

Key words: block cipher; Feistel construction; design for hardware; analysis of security

1 引言

密码技术是保障信息安全的核心技术, 密码算法和密码产品能够自主可控是保证我国信息安全的重要条件. 密码算法是保障信息安全的核心工具和基础支撑, 是解决信息安全问题最有效、最可靠、最经济的手段. 分组密码算法是密码算法中的一个核心分支, 其主要作用是保证信息的机密性和完整性, 在通信系统中起到重要作用. 分组密码算法是将一个明文分组作为整体加密, 并且通常得到的是与明文等长的密文分组, 两个用户之间要共享一个对称加密密钥. 1977 年, DES^[1] 被美国国家标准局采纳为联邦信息处理标准. 2001 年, 美国国家标准技术研究所发布了高级加密标准 AES^[2], 并成为当今主流分组加密算法. 随着人们对分组密码算法的研究更加深入, 越来越多的分组加密算法被提出, 例如 Midori 算法^[3]、Simon 算法^[4]、SPECK 算法^[4] 等等. 分组密码算法的结构主要有三种: Feistel 结构、SPN 结构和 Lai-Massey 结构, 不同的结构具有不同的特点, 因此结构的选取也是设计分组密码算法时首先需要考虑的问题之一. 随着人们对分组密码算法的要求越来越高, 发展方向逐渐偏向轻量级分组密码算法, 例如 2016 年提出的 SKINNY 算法^[5]、2017 年提出的 GIFT 算法^[6] 等等.

Raindrop 算法采用 Feistel 结构, 使得加解密一致, 采取较为简单轮函数, 降低算法硬件实现代价, 同时保证算法的安全性, 旨在设计一个轻量级分组密码算法.

1.1 主要贡献

本文给出了一个面向硬件设计的分组加密算法——Raindrop 算法. 该算法共三个版本: Raindrop-128/128/60、Raindrop-128/256/80 和 Raindrop-256/256/100 (其中 Raindrop- $n/k/R$: n 为分组长度, k 为密钥长度, R 为总轮数). Raindrop 算法主要具有以下几个特点:

(1) 加解密一致, 节省资源

Raindrop 算法采用 Feistel 结构, 加密过程中, 两部分消息状态进行交换, 在最后一轮不交换, 使得加密过程与解密过程一致, 能够节省较多的硬件资源, 不需要为解密算法付出额外代价.

(2) 轮函数简单, 易于轻量化实现

在 Raindrop 算法的非线性运算中, 每个输出比特只需通过 1 个与运算、1 个非运算和 1 个异或运算便可得到, 且线性层的二元域矩阵深度为 1, 因此易于轻量化实现.

(3) 安全性较强

我们利用自动化搜索工具 STP 对 Raindrop 算法进行评估, 主要包括差分分析^[7]、线性分析^[8]、不可能差分分析^[9] 等. 评估结果表明, Raindrop 算法足够抵抗目前已知的攻击.

1.2 本文框架

本文之后内容安排如下: 第 2 节给出 Raindrop 算法描述; 第 3 节给出 Raindrop 算法伪代码; 第 4 节给出 Raindrop 算法设计原理; 第 5 节和第 6 节分别给出 Raindrop 算法的安全性分析和软硬件性能. 最后第 7 节总结全文.

2 Raindrop 算法描述

在描述算法之前, 如下定义本文档中使用的表示符号:

Raindrop- $n/k/R$: 分组长度/密钥长度/算法总轮数的 Raindrop 算法版本
Raindrop- n/k : 分组长度为 n , 密钥长度为 k 的 Raindrop 算法版本
 K : 主密钥
 K_r : 第 r 轮轮密钥, $r \geq 1$
 RF_{64} : 输入为 64 比特的轮函数
 RF_{128} : 输入为 128 比特的轮函数
 S_i : 输入为 i 比特的 S 盒
 M : 行混合操作
 $\lll$: 左循环移位
 $\ggg$: 右循环移位
 $\ll$: 左移位
 $\gg$: 右移位
 $\oplus$: 异或操作

2.1 Raindrop 算法参数

Raindrop 算法共三个版本: Raindrop-128/128、Raindrop-128/256 和 Raindrop-256/256, 每个版本的参数如表 1 所示.

表 1 Raindrop 算法的三个版本的参数设定
Table 1 Three variants of Raindrop block cipher

版本	分组长度 (n)	密钥长度 (k)	总轮数 (R)
Raindrop-128/128	128	128	60
Raindrop-128/256	128	256	80
Raindrop-256/256	256	256	100

2.2 算法结构

Raindrop 算法整体采用 Feistel 结构, 轮函数采用 SP 结构, 包含 S 盒代换、行混合、列循环移位操作. 为了保证加解密的一致性, 最后一轮不进行状态的交换, 轮函数和加密算法的结构见图 1.

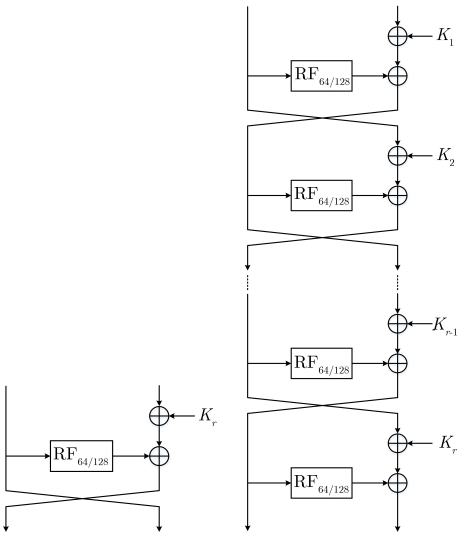


图 1 Raindrop 算法的轮函数和整体结构
Figure 1 Round function and whole construction of Raindrop

2.2.1 消息状态排列

Raindrop 算法采用 Feistel 结构, 包括两部分消息状态. 轮函数的输入可以用一个 4×4 的二维矩阵表示, 称其为状态矩阵.

若轮函数的输入为 64 比特, 表示为 $p_0p_1 \cdots p_{15}$, 可以将其按照如下顺序映射成 4×4 的状态矩阵:

$$\begin{pmatrix} p_0 & p_4 & p_8 & p_{12} \\ p_1 & p_5 & p_9 & p_{13} \\ p_2 & p_6 & p_{10} & p_{14} \\ p_3 & p_7 & p_{11} & p_{15} \end{pmatrix} \quad (1)$$

其中 p_i 的长度 $|p_i|$ 满足: 当 $i = 0, 4, 8, 12, 2, 6, 10, 14$ 时, 有 $|p_i| = 3$; 其余情况下, 有 $|p_i| = 5$.

若轮函数的输入为 128 比特时, 可以用相同的二维矩阵来表示输入状态, p_i 的长度 $|p_i|$ 满足: 当 $i = 0, 4, 8, 12, 2, 6, 10, 14$ 时, 有 $|p_i| = 7$; 其余情况下, 有 $|p_i| = 9$.

每一个 p_i 在本文中被称为一个 word, 因此消息状态可以看成 16 个 word 的组合.

2.2.2 S 盒代换

S 盒代换操作使用不同大小的 S 盒. 其中 Raindrop-128/128、Raindrop-128/256 采用 3 比特和 5 比特 S 盒, Raindrop-256/256 采用 7 比特和 9 比特 S 盒. S 盒的构造使用了 Keccak 函数^[10] 中非线性运算的布尔表达式. 假设 k 比特 S 盒的输入为 $(x_{k-1}, x_{k-2}, \cdots, x_1, x_0)$, 输出为 $(y_{k-1}, y_{k-2}, \cdots, y_1, y_0)$, 则输出比特和输入比特满足如下关系:

$$y_i = x_i \oplus (\sim x_{(i-1) \bmod k}) \& x_{(i-2) \bmod k} \quad (2)$$

2.2.3 行混合

行混合矩阵为:

$$\begin{pmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} \quad (3)$$

此矩阵右乘状态矩阵, 可以得到行混合之后的值, 即:

$$\begin{pmatrix} p_0 & p_4 & p_8 & p_{12} \\ p_1 & p_5 & p_9 & p_{13} \\ p_2 & p_6 & p_{10} & p_{14} \\ p_3 & p_7 & p_{11} & p_{15} \end{pmatrix} \times \begin{pmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} p_4 \oplus p_{12} & p_8 \oplus p_{12} & p_0 & p_0 \oplus p_4 \\ p_5 \oplus p_{13} & p_9 \oplus p_{13} & p_1 & p_1 \oplus p_5 \\ p_6 \oplus p_{14} & p_{10} \oplus p_{14} & p_2 & p_2 \oplus p_6 \\ p_7 \oplus p_{15} & p_{11} \oplus p_{15} & p_3 & p_3 \oplus p_7 \end{pmatrix} \quad (4)$$

2.2.4 列循环移位

记状态矩阵的第 i 列的级联值为 $\text{Col}_i = p_{4i} || p_{(4i+1)} || p_{(4i+2)} || p_{(4i+3)}$, 其中 $0 \leq i \leq 3$.

对 Raindrop-128/128、Raindrop-128/256 算法, 每一列按照下述方式进行比特级的循环移位, 即: Col_0 不变, Col_1 向左循环 6 比特, Col_2 向左循环 7 比特, Col_3 向左循环 12 比特.

对 Raindrop-256/256 算法, 每一列按照下述方式进行比特级的循环移位, 即: Col_0 不变, Col_1 向左循环 12 比特, Col_2 向左循环 14 比特, Col_3 向左循环 24 比特.

2.3 密钥生成算法

Raindrop 算法采用线性操作生成每轮的轮密钥. Raindrop-128/128 主密钥为 128 比特, 每轮异或 64 比特轮密钥, Raindrop-128/256 主密钥为 256 比特, 每轮异或 64 比特轮密钥, Raindrop-256/256 主密钥为 256 比特, 每轮异或 128 比特轮密钥.

2.3.1 Raindrop-128/128 密钥生成算法

假设我们需要生成 R 轮的轮密钥, 那么首先将 128 比特的主密钥赋给密钥状态 TK_1 , 然后按照图 2 所示的方式依次生成密钥状态 TK_r , 其中 r 代表轮数, 满足 $2 \leq r \leq R$, $(r-1) || 0 \cdots 0$ 是

密钥生成算法中使用的轮常数, $0 \cdots 0$ 代表 64 比特的“0”, 即轮常数异或在 TK_{r-1}^3 上, A^4 代表执行连续的 4 次 A 函数. 在执行 A 函数时, 将 128 比特的密钥状态分成 8 块, 每块 16 比特, 即 $TK_r = TK_r^0 || TK_r^1 || TK_r^2 || TK_r^3 || TK_r^4 || TK_r^5 || TK_r^6 || TK_r^7$. A 函数具体的形式见图 3, 注意函数的最后一步是对这 128 比特的状态进行一个整体的左循环移 5 位的操作. 第 r 轮的 64 比特轮密钥 K_r 是 TK_r 的第 64 至 127 比特, 即 $K_r = TK_r[127 : 64]$.

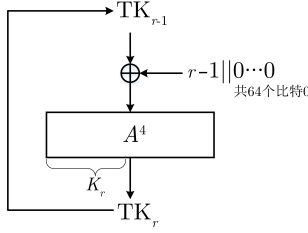


图 2 Raindrop-128/128 的密钥生成算法
Figure 2 Key generation of Raindrop-128/128

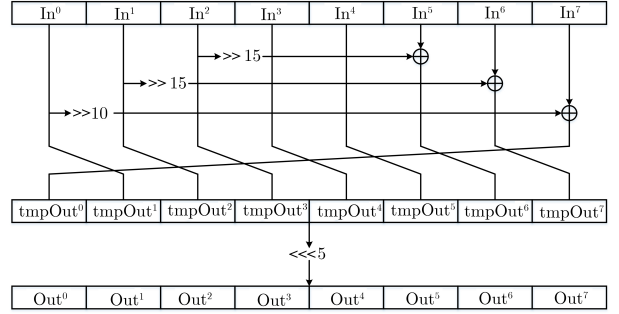


图 3 A 函数
Figure 3 Function A

2.3.2 Raindrop-128/256 密钥生成算法

Raindrop-128/256 算法的密钥生成算法的整体结构与 Raindrop-128/128 类似, 首先将 256 比特的主密钥赋给密钥状态 TK_1 , 然后按照图 4 所示的方式依次生成密钥状态 TK_r , 其中 r 代表轮数, 满足 $2 \leq r \leq R$, $(r-1) || 0 \cdots 0$ 是密钥生成算法中使用的轮常数, $0 \cdots 0$ 代表 160 比特的“0”, 即轮常数异或在 TK_{r-1}^2 上. B^4 代表执行连续的 4 次 B 函数. 在执行 B 函数时, 将 256 比特的密钥状态分成 8 块, 每块 32 比特, 即 $TK_r = TK_r^0 || TK_r^1 || TK_r^2 || TK_r^3 || TK_r^4 || TK_r^5 || TK_r^6 || TK_r^7$. B 函数具体的形式见图 5, 注意函数的最后一步是对这 256 比特的状态进行一个整体的左循环移 13 位的操作. 第 r 轮的 64 比特轮密钥 K_r 是 TK_r 的第 240 至 255 比特, 第 208 至 223 比特, 第 176 至 191 比特, 第 144 至 159 比特的级联值, 即: $K_r = TK_r[255 : 240] || TK_r[223 : 208] || TK_r[191 : 176] || TK_r[159 : 144]$.

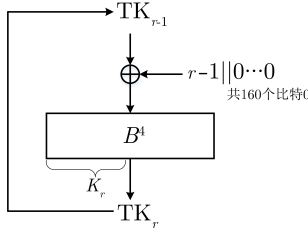


图 4 Raindrop-128/256 的密钥生成算法
Figure 4 Key generation of Raindrop-128/256

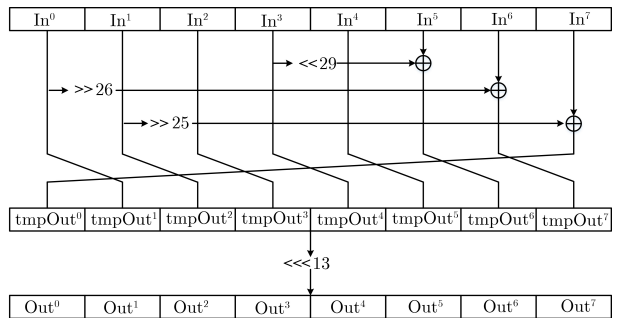


图 5 B 函数
Figure 5 Function B

2.3.3 Raindrop-256/256 密钥生成算法

Raindrop-256/256 的密钥生成算法与 Raindrop-128/256 算法的密钥生成算法相同, 只不过 Raindrop-256/256 使用 128 比特的轮密钥. 第 r 轮的 128 比特轮密钥 K_r 是 TK_r 的第 128 至 255 比特, 即: $K_r = TK_r[255 : 128]$.

3 算法伪代码

3.1 加密算法

假设输入的明文为 $P^L || P^R$, 第 r 轮的轮密钥为 K_r , 加密轮数为 R , 输出的密文为 $C^L || C^R$, S 盒代换操作为 S , 行混合操作为 M , 列循环移位操作为 BR , 则加密算法如算法 1.

算法 1 加密算法

Input: input parameters $P^L || P^R, K_r, R$
Output: output $C^L || C^R$

```

1  $P_0^L || P_0^R = P^L || P^R$ ;
2 for  $0 \leq r < R - 1$  do
3    $P_{r+1}^L = (P_r^R \oplus K_{r+1}) \oplus (BR \circ M \circ S(P_r^L))$ ;
4    $P_{r+1}^R = P_r^L$ ;
5 end
6  $C^L = P_{R-1}^L$ ;
7  $C^R = (P_{R-1}^R \oplus K_R) \oplus (BR \circ M \circ S(P_{R-1}^L))$ ;

```

3.2 S 盒代换

假设 S 盒的输入为 P^L , 输入大小为 n 比特, 输出为 $S(P^L)$, S_i 为 i 比特 S 盒, 则 S 盒代换操作算法如算法 2.

算法 2 S 盒代换

Input: P^L, n
Output: $S(P^L)$

```

1 if  $n = 64$  then
2    $p_0 || p_1 || \dots || p_{15} = P^L$ ;
3   where  $|p_i| = 3$  when  $i = 0, 4, 8, 12, 2, 6, 10, 14$ ;  $|p_i| = 5$  when  $i = 1, 5, 9, 13, 3, 7, 11, 15$ ;
4   for  $i = 0, 4, 8, 12, 2, 6, 10, 14$  do
5      $T_i = S_3(p_i)$ ;
6   end
7   for  $i = 1, 5, 9, 13, 3, 7, 11, 15$  do
8      $T_i = S_5(p_i)$ ;
9   end
10 end
11 if  $n = 128$  then
12    $p_0 || p_1 || \dots || p_{15} = P^L$ ;
13   where  $|p_i| = 7$  when  $i = 0, 4, 8, 12, 2, 6, 10, 14$ ;  $|p_i| = 9$  when  $i = 1, 5, 9, 13, 3, 7, 11, 15$ ;
14   for  $i = 0, 4, 8, 12, 2, 6, 10, 14$  do
15      $T_i = S_7(p_i)$ ;
16   end
17   for  $i = 1, 5, 9, 13, 3, 7, 11, 15$  do
18      $T_i = S_9(p_i)$ ;
19   end
20 end
21  $S(P^L) = T_0 || T_1 || \dots || T_{15}$ ;

```

3.3 行混合

行混合为二元域矩阵右乘状态矩阵, 假设行混合的输入为 P , 行混合的输出为 $M(P)$, 则行混合算法如算法 3.

3.4 列循环移位

列循环移位是状态矩阵的每一列进行循环移位, 假设输入为 P , 输入大小为 n 比特, 输出为 $BR(P)$, 则列循环移位算法如算法 4.

算法 3 行混合

Input: P
Output: $M(P)$

```

1  $p_0 || p_1 || \dots || p_{15} = P$  排列方式如公式 (1);
2 for  $i = 0, 1, 2, 3$  do
3    $T_i = p_{i+4} \oplus p_{i+12}$ ;
4 end
5 for  $i = 4, 5, 6, 7$  do
6    $T_i = p_{i+4} \oplus p_{i+8}$ ;
7 end
8 for  $i = 8, 9, 10, 11$  do
9    $T_i = p_{i-8}$ ;
10 end
11 for  $i = 12, 13, 14, 15$  do
12    $T_i = p_{i-12} \oplus p_{i-8}$ ;
13 end
14  $M(P) = T_0 || T_1 || \dots || T_{15}$ 

```

算法 4 列循环移位

Input: P
Output: $BR(P)$

```

1  $p_0 || p_1 || \dots || p_{15} = P$  排列方式如公式 (1);
2 if  $n = 64$  then
3    $T_0 = p_0 || p_1 || p_2 || p_3$ ;
4    $T_1 = (p_4 || p_5 || p_6 || p_7) \lll 6$ ;
5    $T_2 = (p_8 || p_9 || p_{10} || p_{11}) \lll 7$ ;
6    $T_3 = (p_{12} || p_{13} || p_{14} || p_{15}) \lll 12$ ;
7 end
8 if  $n = 128$  then
9    $T_0 = p_0 || p_1 || p_2 || p_3$ ;
10   $T_1 = (p_4 || p_5 || p_6 || p_7) \lll 12$ ;
11   $T_2 = (p_8 || p_9 || p_{10} || p_{11}) \lll 14$ ;
12   $T_3 = (p_{12} || p_{13} || p_{14} || p_{15}) \lll 24$ ;
13 end
14  $BR(P) = T_0 || T_1 || T_2 || T_3$ ;

```

3.5 Raindrop-128/128 密钥生成算法

设 Raindrop-128/128 轮数为 R , 主密钥为 K , 第 r 轮的轮密钥为 K_r , A 函数为 A , 则 Raindrop-128/128 的密钥生成算法如算法 5.

算法 5 Raindrop-128/128 密钥生成算法

Input: K, R
Output: K_r

```

1  $TK_1 = K$ ;
2 for  $2 \leq r \leq R$  do
3    $TK_r = A \circ A \circ A \circ A(TK_{r-1} \oplus ((r-1) || 000 \dots 000))$ ,  $(r-1)$  共级联了 64 个 0
4 end
5 for  $1 \leq r \leq R$  do
6    $K_r = TK_r[127 : 64]$ ;
7 end

```

3.6 Raindrop-128/256 密钥生成算法

设 Raindrop-128/256 轮数为 R , 主密钥为 K , 第 r 轮的轮密钥为 K_r , B 函数为 B , 则 Raindrop-128/256 的密钥生成算法如算法 6.

算法 6 Raindrop-128/256 密钥生成算法

Input: K, R
Output: K_r

```

1  $TK_1 = K$ ;
2 for  $2 \leq r \leq R$  do
3    $TK_r = B \circ B \circ B \circ B(TK_{r-1} \oplus ((r-1) || 000 \dots 000))$ ,  $(r-1)$  共级联了 160 个 0
4 end
5 for  $1 \leq r \leq R$  do
6    $K_r = TK_r[255 : 240] || TK_r[223 : 208] || TK_r[191 : 176] || TK_r[159 : 144]$ ;
7 end

```

3.7 Raindrop-256/256 密钥生成算法

设 Raindrop-256/256 轮数为 R , 主密钥为 K , 第 r 轮的轮密钥为 K_r , B 函数为 B , 则 Raindrop-256/256 的密钥生成算法见算法 7.

算法 7 Raindrop-256/256 密钥生成算法

Input: K, R
Output: K_r

- 1 $TK_1 = K$;
- 2 **for** $2 \leq r \leq R$ **do**
- 3 $TK_r = B \circ B \circ B \circ B(TK_{r-1} \oplus ((r-1)||000 \cdots 000))$, $(r-1)$ 共级联了 160 个 0
- 4 **end**
- 5 **for** $1 \leq r \leq R$ **do**
- 6 $K_r = TK_r[255 : 128]$;
- 7 **end**

3.8 A 函数

A 函数用于 Raindrop-128/128 密钥生成算法中, 设输入为 PK , 输出为 TK , 则 A 函数的算法见算法 8.

算法 8 A 函数

Input: PK
Output: TK

- 1 $In^0 || In^1 || In^2 || In^3 || In^4 || In^5 || In^6 || In^7 = PK$;
- 2 $tmpOut^0 = In^7 \oplus (In^0 \gg 10)$;
- 3 $tmpOut^1 = In^0$;
- 4 $tmpOut^2 = In^1$;
- 5 $tmpOut^3 = In^2$;
- 6 $tmpOut^4 = In^3$;
- 7 $tmpOut^5 = In^4$;
- 8 $tmpOut^6 = In^5 \oplus (In^2 \gg 15)$;
- 9 $tmpOut^7 = In^6 \oplus (In^1 \gg 15)$;
- 10 $TK = (tmpOut^0 || tmpOut^1 || tmpOut^2 || tmpOut^3 || tmpOut^4 || tmpOut^5 || tmpOut^6 || tmpOut^7) \lll 5$;

3.9 B 函数

B 函数用于 Raindrop-128/256 和 Raindrop-256/256 密钥生成算法中, 设输入为 PK , 输出为 TK , 则 B 函数的算法见算法 9.

算法 9 B 函数

Input: PK
Output: TK

- 1 $In^0 || In^1 || In^2 || In^3 || In^4 || In^5 || In^6 || In^7 = PK$;
- 2 $tmpOut^0 = In^7 \oplus (In^1 \gg 25)$;
- 3 $tmpOut^1 = In^0$;
- 4 $tmpOut^2 = In^1$;
- 5 $tmpOut^3 = In^2$;
- 6 $tmpOut^4 = In^3$;
- 7 $tmpOut^5 = In^4$;
- 8 $tmpOut^6 = In^5 \oplus (In^3 \ll 29)$;
- 9 $tmpOut^7 = In^6 \oplus (In^0 \gg 26)$;
- 10 $TK = (tmpOut^0 || tmpOut^1 || tmpOut^2 || tmpOut^3 || tmpOut^4 || tmpOut^5 || tmpOut^6 || tmpOut^7) \lll 13$;

4 设计原理

在设计 Raindrop 算法时, 主要考虑了硬件实现代价. 设计原理是采用较为简单的非线性层和线性层, 减少硬件面积, 降低关键路径上的时延. 构造异或数较少的线性密钥生成算法, 同时保证密钥生成算法扩

散速度较快.

4.1 简单非线性函数

我们通过 Keccak 非线性运算的布尔表达式来生成 Raindrop 的 4 种 S 盒, 该布尔表达式只包含 1 个与操作, 1 个非操作, 1 个异或操作, 满足硬件面积小, 时延短的要求.

4.2 简单线性层

我们选取一个深度为 1 的二维域矩阵作为线性层的一部分, 深度为 1 保证了在关键路径上最多只贡献 1 个异或, 同时整个矩阵总的异或数较少, 也满足了对硬件实现的要求.

4.3 线性密钥生成算法

对于 Raindrop-128/128 算法生成 1 轮轮密钥总共只需要 32 个异或, 对于 Raindrop-128/256 和 Raindrop-256/256 算法, 生成 1 轮轮密钥只需要 64 个异或, 同时通过筛选密钥生成算法中的移位数、异或位置和循环移位数来达到较好的扩散性.

5 安全性分析

本节给出 Raindrop 算法的差分分析、线性分析、不可能差分分析、零相关分析和积分分析的结果. 鉴于算法结构本身特点, 其它分析方法 (如中间相遇攻击等) 对算法安全性的影响较小, 本文不再给出分析结果.

5.1 差分分析

利用 STP 搜索工具建立自动化搜索模型, 我们发现 Raindrop-128/128、Raindrop-128/256 和 Raindrop-256/256 算法 16 轮最小活跃 S 盒个数均大于等于 32. 根据 S 盒的差分分布表, 我们可知 3 比特、5 比特、7 比特、9 比特 S 盒的非 0 差分传播概率的最大值均为 2^{-2} , 因此 Raindrop-128/128 和 Raindrop-128/256 算法不存在 32 轮及更长轮数的有效差分路线, Raindrop-256/256 算法不存在 64 轮及更长轮数的有效差分路线. 因此 Raindrop 算法能够抵抗差分攻击的.

5.2 线性分析

利用 STP 搜索工具建立自动化搜索模型, 我们发现 Raindrop-128/128、Raindrop-128/256 和 Raindrop-256/256 算法 16 轮最小活跃 S 盒个数均大于等于 32. 根据 S 盒的线性近似表, 我们可知 3 比特、5 比特、7 比特、9 比特 S 盒的非 0 掩码传播相关性的最大值均为 2^{-1} . 因此 Raindrop-128/128 和 Raindrop-128/256 算法不存在 32 轮及更长轮数的有效线性路线, Raindrop-256/256 算法不存在 64 轮及更长轮数的有效线性路线. 因此 Raindrop 算法能够抵抗线性攻击的.

5.3 不可能差分分析

利用 STP 搜索工具建立自动化搜索模型, 我们要求输入差分只有 1 个 p_i (一个 word) 活跃, 输出差分只有 1 个 p_i (一个 word) 活跃. 在这种情况下, Raindrop 算法均不存在 13 轮及以上的 1-1 word 不可能差分. 因此, Raindrop 算法是抵抗 1-1 word 不可能差分攻击的.

5.4 零相关分析

利用 STP 搜索工具建立的自动化搜索模型, 在限定输入掩码和输出掩码各只有一个 p_i (一个 word) 活跃的前提下, Raindrop 算法均不存在 13 轮及以上的 1-1 word 零相关路线. 因此, Raindrop 算法是抵抗 1-1 word 零相关攻击的.

5.5 积分攻击

利用 STP 搜索工具建立的自动化搜索模型, 使用比特级的 Division Property 对 Raindrop 算法进行分析, Raindrop-128/128 和 Raindrop-128/256 算法的积分区分器最长为 7 轮, Raindrop-256/256 积分区分器最长为 13 轮. 因此 Raindrop 算法是抵抗积分攻击的.

6 Raindrop 性能分析

6.1 软件性能分析

本节给出 Raindrop 算法在 64 bit Windows 环境和 32 bit ARM 环境下的实现结果. 测试环境分别为: Intel(R) Core(TM) i7-6700T CPU 2.40 GHz 主频、64 位 Windows10 操作系统、16 GB 内存,

X86-64 环境和基于 STM32 F103 的开发板 (主频 72 Mhz): stm32f1zet6 核心板 6、512 K 片内 Flash、64 K 片内 RAM.

在进行性能测试时, 采用 256 字节数据作为输入, 每次测试变换密钥, 取 10^5 次 CBC 模式运算测试结果的平均值为加/解密速率的结果, 其中加/解密执行时间包含密钥扩展算法运算时间. 测试结果见表 2. 由于 CBC 模式下的解密过程可以并行实现, 我们在表中给出的解密速率是基于多路实现结果. 其中

$$\begin{aligned} \text{加密速率} &= \text{加密数据大小} / \text{加密执行时间}, \\ \text{解密速率} &= \text{解密数据大小} / \text{解密执行时间} \end{aligned}$$

表 2 软件实现结果
Table 2 Results of software implementation

软件实现类别	算法版本	加密速率 (Mbps)	解密速率 (Mbps)
64 bit Windows 环境下 的算法速率优化 C 实现	128/128	242	3126
	128/256	181	2534
	256/256	271	1892
32 bit ARM 环境下的算 法速率优化 C 实现	128/128	1.19	1.19
	128/256	0.91	0.91
	256/256	0.98	0.98

6.2 硬件性能分析

本节给出 Raindrop 算法的 Verilog 硬件仿真实现结果. 采用的仿真工具为 ModelSim SE 10.1a, 综合工具为 Synopsys Design Vision (F-2011.09-SP1 Version), 以及工艺库为 HJTC 0.11 um. 算法 Verilog 硬件仿真实现采用 32 个分组数据进行加/解密. 算法实现的加/解密运算用时包含密钥扩展运算用时. 算法电路面积规模为包含加密、解密、密钥扩展运算的算法整体实现的含互连线面积.

具体方法如下:

(1) 仿真验证

使用 ModelSim 对算法实现正确性进行仿真验证, 并记录 32 个分组数据运算 (含密钥扩展运算) 占用的时钟周期数.

(2) 性能自测试

在给定的时钟约束条件上进行前端综合, 得到并记录以下测试数据:

1) 关键路径时长

在 Synopsys Design Compiler 中进行关键路径时长测试. 根据算法设计提供的自测数据, 设置相应的时钟约束条件, 通过综合软件得到关键路径时长.

2) 速率

在关键路径时长合理的条件下, 通过以下公式计算加解密速率:

$$\text{加密速率} = \text{加密数据长度} / (\text{时钟约束条件} \times \text{加密占用的时钟周期数})$$

$$\text{解密速率} = \text{解密数据长度} / (\text{时钟约束条件} \times \text{解密占用的时钟周期数})$$

3) 面积

在可满足的最小时钟约束条件下, 在 HJTC 0.11 um 工艺库基础上进行综合, 记录算法实现面积 (包含加密、解密、密钥扩展运算的算法整体实现的含互连线面积).

4) 吞面比

$$\text{加密吞面比} = \text{加密速率} / \text{算法电路面积}$$

$$\text{解密吞面比} = \text{解密速率} / \text{算法电路面积}$$

实现结果如表 3 所示.

表 3 Verilog 硬件仿真实验结果
Table 3 Results of Verilog hardware simulation implementation

仿真结果	算法版本		
	Raindrop-128/128	Raindrop-128/256	Raindrop-256/256
运行周期 (Cycles)	3901	5201	6501
时钟约束 (ns)	2.18	2.1	2.2
加密速率 (Mbps)	978.59	761.90	1163.64
解密速率 (Mbps)	948.46	738.54	1128.03
面积 (um ²)	26 672.28	42 137.82	50 238.28
面积 (GE)	2750.00	4344.55	5179.74
加密吞面比 (Mbps/um ²)	0.036 690	0.018 081	0.023 162
解密吞面比 (Mbps/um ²)	0.035 560	0.017 527	0.022 454

7 总结

本文给出了一个新的分组加密算法——Raindrop 算法. Raindrop 算法采用 Feistel 结构. 轮函数为 SP 结构, 采用不同大小 S 盒进行组合, S 盒的布尔表达式只包含 1 个与操作、1 个非操作、1 个异或操作. 轮函数的行混合操作使用深度为 1 的二元域矩阵. 密钥生成算法使用异或数较少, 扩散性较强的线性操作. 通过以上特点可以看出, 轮函数容易进行门限实现, 且实现代价很小, 从而保证了 Raindrop 算法在硬件实现上具有较好的优势. 此外, 本文通过利用自动化搜索工具 STP 对 Raindrop 算法的安全性进行评估, 评估结果显示 Raindrop 算法可以抵抗差分攻击、线性攻击等已知攻击, 具有较好的安全冗余.

但由于 Raindrop 算法采用了 3、5、7、9 比特 S 盒, 使得软件实现较为繁琐, 同时由于 Raindrop 算法的独特性, 在安全性分析时, 主要对活跃 S 盒的个数进行评估, 最终使得 Raindrop 算法的安全界较为宽松, 因此降低 Raindrop 算法的轮数仍有很大潜力.

References

[1] D. E. Standard. Data encryption standard. Federal Information Processing Standards Publication, 1999.

[2] DAEMEN J, RIJMEN V. AES proposal: Rijndael[OL]. 1999. https://cs.ru.nl/~joan/papers/JDA_VRI_Rijndael_V2_1999.pdf

[3] BANIK S, BOGDANOV A, ISOBE T, et al. Midori: A block cipher for low energy[C]. In: Advances in Cryptology—ASIACRYPT 2015. Springer Berlin Heidelberg, 2015: 411–436. [DOI: 10.1007/978-3-662-48800-3_17]

[4] BEAULIEU R, SHOR D, SMITH J, et al. The SIMON and SPECK lightweight block ciphers[C]. In: Proceedings of the 52nd Annual Design Automation Conference. ACM, 2015: Article No. 175. [DOI: 10.1145/2744769.2747946]

[5] BEIERLE C, JEAN J, KÖLBL S, et al. The SKINNY family of block ciphers and its low-latency variant MANTIS[C]. In: Advances in Cryptology—CRYPTO 2016, Part II. Springer Berlin Heidelberg, 2016: 123–153. [DOI: 10.1007/978-3-662-53008-5_5]

[6] BANIK S, PANDEY S K, PEYRIN T, et al. GIFT: A small present[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2017. Springer Cham, 2017: 321–345. [DOI: 10.1007/978-3-319-66787-4_16]

[7] BIHAM E, SHAMIR A. Differential cryptanalysis of DES-like cryptosystems[C]. In: Advances in Cryptology—CRYPTO '90. Springer Berlin Heidelberg, 1990: 2–21. [DOI: 10.1007/3-540-38424-3_1]

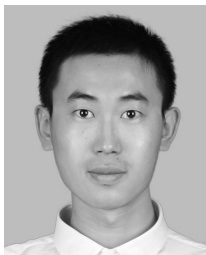
[8] MATSUI M. Linear cryptanalysis method for DES cipher[C]. In: Advances in Cryptology—EUROCRYPT '93. Springer Berlin Heidelberg, 1993: 386–397. [DOI: 10.1007/3-540-48285-7_33]

[9] BIHAM E, BIRYUKOV A, SHAMIR A. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials[C]. In: Advances in Cryptology—EUROCRYPT '99. Springer Berlin Heidelberg, 1999: 12–23. [DOI: 10.1007/3-540-48910-X_2]

[10] BERTONI G, DAEMEN J, PEETERS M, et al. The Keccak reference[OL]. In: Submission to NIST (Round 3),

2011, 13: 14–15. <https://keccak.team/files/Keccak-reference-3.0.pdf>

作者信息



李永清 (1995–), 河北廊坊人, 硕士在读. 主要研究领域为对称密码算法的设计和分析.
yqli@mail.sdu.edu.cn



李木舟 (1995–), 山东济宁人, 博士在读. 主要研究领域为对称密码算法设计和分析.
muzhouli@mail.sdu.edu.cn



付勇 (1981–), 山东寿光人, 副研究员. 主要研究领域为密码工程.
yongfu0976@163.com



樊艳红 (1979–), 山东聊城人, 博士在读. 主要研究领域为对称密码算法的安全性分析和实现.
fanyh@mail.sdu.edu.cn



黄鲁宁 (1979–), 山东海阳人, 助理研究员. 主要研究领域为对称密码算法的安全性分析.
lnhuang@sdu.edu.cn



王美琴 (1974–), 宁夏银川人, 教授. 主要研究领域为对称密码算法的设计和分析.
mqwang@sdu.edu.cn